Computer Science Portfolio: Design, Implementation, and Reflection

Patrick Thompson
Student ID: GH1057420
B201 Computer Science Lab
GISMA University of Applied Sciences

June 17, 2026

Contents

1 Introduction

This report documents the design and development of my computer science portfolio, created as part of the B201 Computer Science Lab module at GISMA University of Applied Sciences. The portfolio is a personal website, hosted on GitHub Pages, that presents my skills, projects, and credentials to a specific audience: hiring managers recruiting for working student (Werkstudent) cloud engineering positions in Germany.

I chose that audience deliberately because it shaped every decision I made. When I looked through real job listings on platforms such as Hays and StepStone (Hays, 2024), the same requirements appeared again and again: AWS, Terraform or other Infrastructure as Code tooling, Docker, CI/CD pipelines, Linux fundamentals, and a candidate who can document and explain what they have built. A portfolio aimed at that market therefore cannot simply list technologies. It has to prove, with evidence, that I can build, automate, secure, monitor, and clearly communicate cloud infrastructure.

The live portfolio is available at pattson22.github.io, and the source code, along with my CV, technical exercises, and this report, lives in the GitHub repository at github.com/pattson22/pattson22.github.io.

This report covers the full project. I begin by describing the structure of both the website and the repository, and I justify the design decisions behind them. I then explain the tools and technologies I used and why I selected each one. After that I describe the portfolio's content, including my five featured projects and three technical exercises. The most important part, for me, is the critical evaluation, where I assess honestly what works well and what does not. I close with a set of concrete future improvements and a short reflection on what the project taught me and how it prepares me for a role in industry.

2 Portfolio Design and Structure

2.1 Website Structure

The website is a single-page application divided into clearly labelled sections that a visitor scrolls through in a deliberate order: a hero introduction, an "About" section, certifications, featured projects, a live GitHub feed, technical exercises, and contact details. I designed the order around how a busy recruiter actually reads a portfolio.

The hero section states who I am and what I do in one glance, with a short animated terminal that signals my comfort in a command-line, infrastructure context. The "About" section then gives the human story and frames my transition from operations leadership into cloud engineering. Certifications come next, because a verified AWS Solutions Architect credential is an immediate trust signal for a cloud role. Only then do I present the projects, which are the core evidence. I placed the curated "Featured Projects" above the

dynamic GitHub feed on purpose: the curated cards tell the reader exactly what I built and why it matters, while the live feed proves the work is real and ongoing. The technical exercises follow, reassuring an employer about the fundamentals, and the contact section closes with a clear call to action.

2.2 GitHub Repository Structure

The repository is organised so that anyone can navigate it without explanation. The website files (`index.html`, `style.css`, `script.js`) sit at the root, as GitHub Pages expects. Supporting material is grouped into purpose-named folders: `/assets` for images and architecture diagrams, `/cv` for my LaTeX CV and its compiled PDF, `/exercises` for the three technical demonstrations (each in its own subfolder with a script and a README), and `/report` for this document. A single top-level `README.md` ties everything together.

I organised it this way because repository structure is itself a piece of evidence. A cloud engineer spends a large part of their day reading and writing documentation and keeping infrastructure code tidy, so a clean, predictable layout demonstrates exactly the discipline the role demands. Each exercise folder carrying its own README mirrors how real infrastructure repositories are documented module by module.

2.3 Design Decisions

Several decisions defined the final result, and each was a trade-off I made consciously.

I chose a **single-page layout** because recruiters typically spend very little time on any one portfolio. A single, scannable page removes navigation friction and lets a reader reach the projects in seconds. I added a **dark and light mode toggle** for accessibility and personal preference; the choice is saved to `localStorage` so a returning visitor keeps their setting. I selected an **AWS-orange accent colour** deliberately, because it visually ties the whole site to the cloud platform I specialise in and reinforces my positioning the moment the page loads. I hosted on **GitHub Pages** because it is free, integrates directly with the repository that already holds my code, and is an industry-standard way for developers to publish a portfolio. Finally, I built the site to be fully **responsive**, since a hiring manager may well open it on a phone, and a layout that breaks on mobile would undermine the very competence I am trying to demonstrate.

3 Tools and Technologies

I kept the technology stack deliberately lean, choosing tools that match the job rather than the most fashionable option.

HTML5 provides the semantic structure of the site; I used meaningful elements and section landmarks so the page is accessible and easy to read. **CSS3** handles all styling,

and I leaned on CSS custom properties (variables) to implement theming cleanly, so the dark/light switch only has to swap a small set of values. I used Flexbox and CSS Grid for layout because they make responsive design straightforward without a framework.

JavaScript powers the interactive behaviour: it toggles and persists the theme, animates sections into view as the user scrolls, and fetches my public repositories from the GitHub REST API so the "Live from GitHub" section stays current without any manual editing. I chose vanilla JavaScript rather than a framework because the site is small, and avoiding a build step keeps it fast and easy to host directly on GitHub Pages.

LaTeX, via Overleaf, produced both my CV and this report. I chose LaTeX because it gives precise, professional, and consistent typesetting, and Overleaf removes any local installation overhead. **Git and GitHub** provide version control and hosting; using them is not just convenient but is itself evidence of a core professional skill. Finally, I used **draw.io** to create the architecture diagrams for my two flagship cloud projects, because a clear diagram communicates a system design far faster than a paragraph of prose.

4 Portfolio Content

4.1 Featured Projects

I curated five projects and ordered them by how directly they map onto German cloud engineering requirements. My flagship is the **Multi-tier Cloud Application**, a three-tier AWS environment provisioned with Terraform, containerised with Docker, and deployed through a GitHub Actions pipeline. I lead with it because end-to-end cloud deployment is the single capability employers ask for most. Second is the **Highly Available Crypto API**, which demonstrates designing for failure across multiple Availability Zones with an Application Load Balancer and Auto Scaling. Third, the **MLOps API Infrastructure** shows I can deploy and serve applications in a reproducible, containerised way. Fourth, the **Group Chat Application** broadens the picture to full-stack development and real-time concurrency. Fifth, the **Simple Shell** written in C proves low-level Linux and operating-system understanding, which underpins everything that runs in the cloud.

Each project card describes what the system does, lists its technology stack, links to the source on GitHub, and states one concrete thing I learned. For the two cloud projects I also drew architecture diagrams, because being able to visualise and explain a system is a differentiator that many junior portfolios lack.

4.2 Technical Exercises

Alongside the larger projects I added three small, focused exercises that target the fundamentals German employers test for. The first is a **Bash automation script** that idempotently bootstraps a server and runs health checks, returning a non-zero exit code

on failure so it can be used in automation. The second is a **Python script using boto3** that audits an AWS account, flagging S3 buckets that are unencrypted or publicly accessible. The third is a **networking walkthrough** documenting how I build a VPC with public and private subnets, routing, gateways, and least-privilege security groups. Together these prove I am comfortable with scripting, the AWS SDK, security awareness, and core networking, not only with higher-level tooling.

4.3 Curriculum Vitae

My CV was written in LaTeX and is available from the portfolio as a downloadable PDF, with both the source and the compiled file kept in the repository. It highlights my AWS certification, my Infrastructure as Code and CI/CD experience, and the same projects shown on the site, so the website and the CV tell one consistent story.

5 Critical Evaluation

5.1 Strengths

The portfolio has clear strengths. The **dynamic GitHub integration** means the live projects section updates itself whenever I push new work, so the site never becomes stale without my attention. The **design is clean and fully responsive**, working consistently across desktop and mobile, which matters because I cannot control the device a recruiter uses. The **cloud-focused branding** is deliberate and coherent: the AWS-orange accent, the terminal motif, and the project selection all signal the same target role, so there is no ambiguity about what I do. The **repository is well organised and documented**, which is itself evidence of the operational discipline a cloud team needs. Finally, the **architecture diagrams** demonstrate system thinking and the ability to communicate a design, not just to write code.

5.2 Weaknesses

Being honest about the gaps matters more than listing strengths, and there are several. The site has **limited interactivity**: there is no contact form, so a visitor has to switch to email rather than reaching me in place. The **project descriptions, while specific, could go deeper**; a full case study for each one, with the problems I hit and how I solved them, would be more persuasive than a paragraph. I currently display **only one certification**, which understates ongoing learning. There is **no blog or technical writing section**, which is a missed opportunity to show communication skill and depth. The site has **no visitor analytics**, so I cannot tell whether anyone engages with it or where they drop off. And although I am targeting the German market, the portfolio exists **only in**

English; a German-language version would widen its reach and signal commitment to working in Germany. I see these not as failures but as a concrete, prioritised backlog.

6 Future Improvements

Several improvements follow directly from that evaluation. In the near term I would **add a working contact form** so visitors can reach me without leaving the page, and **integrate lightweight visitor analytics** to learn which sections hold attention. I plan to **add further certifications**, beginning with the HashiCorp Terraform Associate and later an AWS Developer credential, to show momentum. To demonstrate communication depth I would **start a short technical blog** documenting things I have built and problems I have solved. I would also **expand the projects into detailed case study pages** with screenshots and decision logs, and **add a German language toggle** to better serve the local market. Taken together these turn a strong static portfolio into a living, growing professional presence.

7 Conclusion

Building this portfolio taught me that presenting technical work well is a discipline in its own right. The hardest part was not the code but the editing: deciding what to show, in what order, and how to explain it so that a busy reader immediately understands my value. Working through the design, documentation, and writing made me far more deliberate about how I communicate engineering decisions. The result is a focused, honest, and well-organised portfolio that demonstrates the exact skills a working student cloud engineering role in Germany demands, and the critical evaluation gives me a clear roadmap for improving it further. More than a coursework deliverable, it is a professional asset I will keep developing as I move into industry.

References

- Amazon Web Services (2024) *AWS Well-Architected Framework*. Available at: https://aws.amazon.com/architecture/well-architected/ (Accessed: 17 June 2026).
- GitHub (2024) *GitHub Pages Documentation*. Available at: https://docs.github.com/en/pages (Accessed: 17 June 2026).
- Hays (2024) *IT & Cloud Engineering Jobs in Germany*. Available at: https://www.hays.de/en/jobs/it (Accessed: 17 June 2026).
- HashiCorp (2024) *Terraform Documentation*. Available at: https://developer.hashicorp.com/terraform/docs (Accessed: 17 June 2026).
- JGraph (2024) *draw.io (diagrams.net) Documentation*. Available at: https://www.drawio.com/doc/ (Accessed: 17 June 2026).
- Mozilla (2024) *MDN Web Docs: HTML, CSS and JavaScript references*. Available at: https://developer.mozilla.org/ (Accessed: 17 June 2026).
- Overleaf (2024) *Documentation and Learn LaTeX*. Available at: https://www.overleaf.com/learn (Accessed: 17 June 2026).
- StepStone (2024) *Cloud Engineer Werkstudent positions, Germany*. Available at: https://www.stepstone.de/ (Accessed: 17 June 2026).